

Cheap Digit Decomposition and Large Plaintext Spaces in FHEW using Phase Splitting

Leonard Schild¹[0009–0007–3336–1258], Aysajan Abidin¹[0000–0002–5128–3608], and Bart Preneel¹[0000–0003–2005–9651]

COSIC KU Leuven, Leuven 3000, Belgium
firstname.lastname [at] esat.kuleuven.be

Abstract. Fully homomorphic encryption algorithms enable users to perform computation on encrypted data. Since the first candidate scheme was proposed in 2009 by Gentry [17], schemes have rapidly improved in all metrics, be it computational complexity or memory efficiency. The class of accumulator based schemes which include FHEW [15] and TFHE [8] are designed to operate on small data, usually ranging between 4 and 5 bits. Yet, schemes can be effectively leveraged in practice and enable lifting of small data to larger plaintext domains through the use of so-called programmable bootstrapping, the ability to evaluate arbitrary functions on an encrypted datum with time independent of the function.

In this work, we present novel methods for homomorphic digit decomposition, the task of efficiently breaking up a large encrypted datum, vastly exceeding the conventional plaintext domain size, into a radix representation for a chosen basis. Our approach relies on a computationally inexpensive decomposition of a ciphertext into chunks that can be assembled into the original message, *without* requiring that such chunks correspond to actual digits. Asymptotically, our approach doubles the performance compared to prior work and practically is 90% faster than the state of the art by Liu et al. [28]. As a direct consequence of our digit decomposition, we describe how to increase the size of the plaintext domain by a large factor, while only doubling the computational complexity and not causing a super-polynomial slowdown. Although concurrent works on functional bootstrapping reach similar improvements regarding the plaintext domain, our approach shines through its conceptual simplicity and flexibility.

Keywords: Fully Homomorphic Encryption, Digit Decomposition, FHEW

1 Introduction

Fully Homomorphic Encryption (FHE) enables computation directly on encrypted data, ensuring that sensitive information remains confidential throughout the processing pipeline. Since Gentry’s seminal work in 2009 introducing the first FHE scheme [17], significant progress has been made to improve efficiency, noise management, and bootstrapping performance [8,9,7]. FHE have evolved into practical cryptographic tools for privacy-preserving analytics, federated learning, and secure cloud computation [23].

Accumulator-based schemes, such as FHEW [15] and TFHE [8,9], have emerged as particularly efficient for Boolean and small-integer computations. Their efficiency stems from fast bootstrapping procedures that refresh ciphertext noise, enabling sub-millisecond gate

evaluation. Although these schemes operate over small plaintext space—typically messages of length 4–5 bits—the mechanism of programmable bootstrapping allows them to evaluate arbitrary functions on encrypted inputs, thereby enabling computation of complex functions despite their small plaintext space [11]. An often overlooked feature of programmable bootstrapping is an implicit rescaling functionality, which allows such encrypted data to be lifted into larger plaintext space. Such rescaling is the primary mechanism enabling large-precision homomorphic digit decomposition procedures, which expresses encrypted values in a structured radix representation.

In this work, we revisit digit decomposition from a different perspective. We introduce phase splitting or phase decomposition, a lightweight technique that decomposes the encrypted phase into structured components that need not correspond to canonical radix digits. Although the space of resulting “pseudo-digits” is not isomorphic or even strictly related to the original ciphertext space, we show that obtaining the canonical radix digits becomes an efficient step from this perspective. Our approach achieves asymptotically a twofold speedup over the state of the art, and up to a 90% improvement in practical implementations, outperforming the prior decomposition techniques [28]. Beyond these performance gains, phase splitting naturally enables an expanded plaintext domain: we demonstrate that the plaintext modulus can be increased by a large factor while incurring only a doubling of computational complexity and allowing re-use of parameters for small spaces. This trade-off significantly enhances the expressive capacity of accumulator-based FHEW-like schemes without proportionally increasing the cost. While recent works on functional and generalized bootstrapping provide alternative means of enlarging plaintext space [31], our approach distinguishes itself through its conceptual simplicity, modularity, and compatibility with existing accumulator structures. Phase splitting requires no new cryptographic assumptions and integrates cleanly into current system designs, providing a practical and implementation-friendly path toward supporting larger plaintext domains in FHEW-style schemes.

Our contributions:

- *Digit-decomposition in the exponent.* We introduce a decomposition methodology which is carried out via exact arithmetic in the exponent of an encrypted polynomial. By enforcing the use of exact arithmetic, we remove traditional constraints regarding rounding errors related to modulus switching or other operations. Our approach is twice as fast asymptotically and may be used under weaker requirements than prior work, including allowing the use of a ciphertext modulus exceeding the FHEW ring modulus.
- *Large plaintext space homomorphic function evaluation.* Based on the digit-decomposition methodology, we demonstrate how to extend the plaintext space for functional bootstrapping in which a negacyclic function $f(x) = -f(x + N) \pmod{2N}$ is evaluated homomorphically. Practically, our approach yields a growth of more than $2\times$ in terms of the plaintext space size, without requiring other values parameterizing the FHEW schemes to be adapted.
- *Half-ring, full-domain bootstrapping.* As a final contribution, we show that applying our large plaintext space functional bootstrapping technique to a plaintext space $\mathbb{Z}_t$ with small size $t < 2N$ immediately enables us to evaluate not just negacyclic, but arbitrary functions.

2 Preliminaries

2.1 Notation

Given $a, b \in \mathbb{Z}$, let $[a, b]$ denote the set $\{a, a+1, \dots, b\}$, and let $[a]$ denote $[1, a]$. For an integer q , let $\mathbb{Z}_q$ denote the ring of integers modulo q . $x \pmod{q}$ will denote the modular reduction by q into $[0, q) \cap \mathbb{Z}$ and by $[x]_q$ symmetric reduction into $(-\frac{q+1}{2}, \frac{q+1}{2}) \cap \mathbb{Z}$. For a power-of-two N , let $\mathcal{K} = \mathbb{Q}[X]/(X^N + 1)$, $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$ and $\mathcal{R}_q = \mathcal{R}/(q\mathcal{R})$. We use lowercase bold such as $\mathbf{u}$ for (row) vectors and uppercase bold such as $\mathbf{U}$ for matrices. The inner product between two vectors $\mathbf{u}$ and $\mathbf{v}$ is denoted as $\langle \mathbf{u}, \mathbf{v} \rangle$. We use notations such as $\mathbf{b}$ or $b(X)$ to denote elements of a polynomial ring. The operator $\lfloor \cdot \rfloor$ denotes rounding, which extends coefficient-wise to a polynomial. Given a probability distribution χ , let $a \leftarrow \chi$ denote that a is sampled from χ . Given a set S , let $a \xleftarrow{\$} S$ denote that a is sampled uniformly random from S .

2.2 Learning with errors and ring learning with errors

The security of our methods and used FHE schemes rely on the hardness of the learning with errors (LWE) problem [34] and the ring learning with errors (RLWE) problem [29,30].

LWE The LWE problem is parametrized by a dimension n , integer modulus q , a key distribution χ_{key} over $\mathbb{Z}$, and an error distribution χ over $\mathbb{Z}$.

Definition 1 (LWE distribution $\mathcal{D}_{\mathbf{s}, \chi}$). Given $\mathbf{s} \in \mathbb{Z}_q^n$, the LWE distribution $\mathcal{D}_{\mathbf{s}, \chi}$ over $\mathbb{Z}_q^n \times \mathbb{Z}_q$ is sampled by choosing a uniformly random $\mathbf{a} \in \mathbb{Z}_q^n$ and error $e \leftarrow \chi$, and outputting $(\mathbf{a}, b = \langle \mathbf{a}, \mathbf{s} \rangle + e \pmod{q}) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$.

We can represent $m \geq 1$ LWE instances with the same secret $\mathbf{s} \in \mathbb{Z}_q^n$ in the matrix form $(\mathbf{A}, \mathbf{b} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e} \pmod{q})$, where $\mathbf{A} \in \mathbb{Z}_q^{m \times n}$ and $\mathbf{b}, \mathbf{e} \in \mathbb{Z}_q^m$. Below we describe two versions of the LWE problem, both conjectured to be hard for appropriately chosen parameters. In practice, concrete security estimates of LWE instances are typically estimated using Albrecht et al.'s lattice estimator [2].

Definition 2 (Search LWE). Given m LWE instances $(\mathbf{A}, \mathbf{b} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e} \pmod{q})$ sampled from $\mathcal{D}_{\mathbf{s}, \chi}$ for some $\mathbf{s} \leftarrow \chi_{key}$, the search LWE problem is to recover $\mathbf{s}$.

Definition 3 (Decision LWE). The decision LWE problem is to distinguish m LWE instances $(\mathbf{A}, \mathbf{b} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e} \pmod{q})$ sampled from $\mathcal{D}_{\mathbf{s}, \chi}$ with respect to $\mathbf{s} \leftarrow \chi_{key}$, from $(\mathbf{A}, \mathbf{b}) \xleftarrow{\$} \mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m$.

RLWE The RLWE problem extends the LWE problem to structured algebraic rings. This work only focuses on the ring $\mathcal{R}$, a cyclotomic ring with a power-of-two cyclotomic order.

Definition 4 (RLWE distribution $\mathcal{D}_{\mathbf{s}, \chi}$). Given $\mathbf{s} \in \mathcal{R}_q$ and an error distribution χ over $\mathcal{R}_q$, the RLWE distribution $\mathcal{D}_{\mathbf{s}, \chi}$ over $\mathcal{R}_q^2$ is sampled by choosing a uniformly random $\mathbf{a} \in \mathcal{R}_q$ and an error $\mathbf{e} \leftarrow \chi$, and outputting $(\mathbf{a}, \mathbf{b} = \mathbf{a} \cdot \mathbf{s} + \mathbf{e} \pmod{q}) \in \mathcal{R}_q^2$.

Then we describe the two versions of the RLWE problem. To date, no known attack on RLWE problems exploits its additional ring structure compared to LWE. Therefore, the security of an RLWE instance is estimated by transforming it to a corresponding LWE instance with the same dimension, modulus, and an appropriate error distribution. Let χ_{key} denote a key distribution over the ring $\mathcal{R}_q$.

Definition 5 (Search RLWE). *Given m RLWE instances $\{(\mathbf{a}_i, \mathbf{b}_i = \mathbf{a} \cdot \mathbf{s} + \mathbf{e} \bmod q)\}_{i \in [m]}$ sampled from $\mathcal{D}_{\mathbf{s}, \chi}$ for some $\mathbf{s} \leftarrow \chi_{key}$, the search RLWE problem is to recover $\mathbf{s}$.*

Definition 6 (Decision RLWE). *The decision RLWE problem is to distinguish m RLWE instances $\{(\mathbf{a}_i, \mathbf{b}_i = \mathbf{a} \cdot \mathbf{s} + \mathbf{e} \bmod q)\}_{i \in [m]}$ sampled from $\mathcal{D}_{\mathbf{s}, \chi}$ with respect to $\mathbf{s} \leftarrow \chi_{key}$, from m uniformly sampled pairs $\{(\mathbf{a}_i, \mathbf{b}_i) \xleftarrow{\$} \mathcal{R}_q^2\}_{i \in [m]}$.*

2.3 FHE primitives

Our contributions adopt a cross-scheme approach, and leverages concepts from a range of FHE schemes such as GSW [19], FHEW [16] and TFHE [10], along with state-of-the-art optimizations [18, 22, 6, 11, 33, 28, 13, 35]. This subsection outlines the relevant FHE ciphertext types, basic operations and subroutines.

FHE ciphertexts Let t denote a plaintext modulus, $Q, q \gg t$ denote ciphertext moduli, and N the dimension of $\mathcal{R}_q$

- $\text{LWE}_{\mathbf{s}}^{n,q}(m) = (\mathbf{a}, b) \in \mathbb{Z}_q^{n+1}$ denotes an LWE ciphertext that satisfies

$$b = \langle \mathbf{a}, \mathbf{s} \rangle + \tilde{m} + e \bmod q,$$

where $\mathbf{s} \in \mathbb{Z}_q^n$ is the secret key, $\tilde{m} \in \mathbb{Z}_q$ is (the encoding of) a message, and $e \in \mathbb{Z}_q$ is an error in a pre-determined error distribution, commonly a (sub)gaussian distribution. In general, $\tilde{m} = \frac{q}{t}m$ to separate the message $m \in \mathbb{Z}_t$ and error term e , and enable correct decoding provided $|e| \leq \frac{q}{2t}$. Furthermore, $\varphi(\text{LWE}_{\mathbf{s}}^{n,q}(m))$ refers to the *phase* or decryption of a LWE sample equal to $b - \langle \mathbf{a}, \mathbf{s} \rangle$.

- $\text{RLWE}_{\mathbf{s}}^{N,Q}(\mathbf{m}) = (\mathbf{a}, \mathbf{b}) \in \mathcal{R}_Q^2$ denotes an RLWE ciphertext that satisfies

$$\mathbf{b} = \mathbf{a} \cdot \mathbf{s} + \tilde{\mathbf{m}} + \mathbf{e} \bmod Q,$$

where $\mathbf{s} \in \mathcal{R}$ is the secret key, $\tilde{\mathbf{m}} \in \mathcal{R}_Q$ is (the encoding of) a message, and $\mathbf{e} \in \mathcal{R}$ is an error in a pre-determined error distribution. We stress that in general $Q \neq q$ and once again we commonly have that $\tilde{\mathbf{m}} = \frac{Q}{t}\mathbf{m}$, $\mathbf{m} \in \mathcal{R}_t$, though not always like in the CKKS FHE scheme [7].

For both LWE and RLWE sample we omit the parameters unless necessary. Furthermore, both LWE and RLWE samples are instances of *generalized* LWE samples and both enable homomorphic addition and scalar multiplication by default via component-wise operations. Finally, we note that the subprocedures we introduce next implicitly rely on extended ciphertext types (e.g. RGSW [21] samples which can be viewed as matrices of RLWE samples) but choose not to describe them in detail in order to simplify the exposition.

FHE basic operations Fully homomorphic encryption allows computation over ciphertexts. The basic homomorphic operations used in this work are as follows.

- **Add** denotes the homomorphic addition between (i) two ciphertexts of the same type (e.g. LWE or RLWE) encrypted under the same secret key, or (ii) one ciphertext and one plaintext. We represent addition of ciphertexts with the usual $+$.
- **ScalarMul** denotes homomorphic scalar multiplication. In the case of LWE samples, this corresponds to multiplication by an element of $\mathbb{Z}_q$ and for RLWE with an element of $\mathcal{R}_Q$. We represent scalar multiplication with the $\cdot$ operator
- **Aut $_{\sigma_i}$** (acc, autK) denotes the homomorphic automorphism that maps $\text{acc} = \text{RLWE}(\mathbf{m})$ to $\text{RLWE}(\sigma_i(\mathbf{m}))$ using an automorphism key autK, where $\sigma_i : \mathbf{m}(X) \rightarrow \mathbf{m}(X^i)$ denotes an automorphism in the Galois group $\text{Gal}(\mathcal{K}/\mathbb{Q})$ for $\gcd(i, N) = 1$.
- **BlindRotate**(acc, ct, bsk) denotes the blind rotation operation, which takes $\text{ct} = [\mathbf{a}, b] = \text{LWE}_{\mathbf{s}}^{n,q}(\tilde{m})$, a *bootstrapping key* bsk, and the accumulator $\text{acc} = \text{RLWE}(\mathbf{p})$ as inputs. The typical LWE modulus q is N or $2N$, and let the *phase* $\varphi(\text{ct}) := b - \langle \mathbf{a}, \mathbf{s} \rangle$. Then the output is $\text{RLWE}(\mathbf{p} \cdot X^{\varphi(\text{ct}) \bmod 2N})$ with a constant noise level independent of ct.
- **SampleExtract**($\text{RLWE}_{\mathbf{s}}^{N,Q}(\mathbf{m}), k$) $\rightarrow \text{LWE}_{\mathbf{s}}^{N,Q}(m_k)$ denotes the sample extraction taking $\text{RLWE}_{\mathbf{s}}^{N,Q}(\mathbf{m})$ and an index $k \in [0, N-1]$ as inputs, and outputting $\text{LWE}_{\mathbf{s}}^{N,Q}(m_k)$ where m_k is the k -th coefficient of $\mathbf{m}$ and $\mathbf{s}$ is the coefficient vector of the RLWE secret $\mathbf{s}$.

Furthermore, we recall FHE procedure that we mention in our work, but do not use:

- **ModSwitch $_{q \rightarrow q'}$** (ct) denotes the modulus switching operation (i) from $\text{LWE}_{\mathbf{s}}^{n,q}(m)$ to $\text{LWE}_{\mathbf{s}}^{n,q'}(m)$ for $m \in \mathbb{Z}_q$, or (ii) from $\text{RLWE}_{\mathbf{s}}^{N,Q}(\mathbf{m})$ to $\text{RLWE}_{\mathbf{s}}^{N,Q'}(\mathbf{m})$ for $\mathbf{m} \in \mathcal{R}_Q$.
- **KeySwitch $_{\mathbf{s} \rightarrow \mathbf{s}'}$** (ct, ksK) denotes the key switching operation mapping a $\text{ct} = \text{LWE}_{\mathbf{s}}^{n,q}(\tilde{m})$ to $\text{ct}' = \text{LWE}_{\mathbf{s}'}^{n,q}(\tilde{m})$, a ciphertext encoding the same message under a different LWE key $\mathbf{s}'$ using an key-switching key ksK.

Finally, throughout this work we will omit the mention of automorphism, bootstrapping and key-switching keys to keep the exposition concise as their internal structure does not affect the correctness of our approaches. For concrete details, we refer to Appendix A-1 [23]

3 Digit Decomposition

In this section we describe our novel digit decomposition approach and, as a result of the latter, high precision functional or programmable bootstrapping. Before, describing our contribution, we briefly recall how the FHE subprocedures previously introduced can be leveraged to evaluate certain functions homomorphically. This process is commonly referred to as programmable or functional bootstrapping.

We consider a ciphertext $\text{ct} = \text{LWE}_{\mathbf{s}}^{n,q}(\tilde{m})$, $\tilde{m} = \frac{q}{t} \cdot m + e$ for $0 \leq m < t$. Let us assume that $q = 2N$ for the ring dimension N of $\mathcal{R}_Q$ and $Q \gg q$. Furthermore, let $|e| \leq \frac{q}{2}$ and let $f : \mathbb{Z}_{2N} \mapsto \mathbb{Z}_{2N}$ be a *negacyclic* function, that is, $f(x + N) = -f(x) \pmod{2N}$.

As an example, consider an encoding in which the most significant bit of a value $x \in \mathbb{Z}_{2N}$ corresponds to the sign ± 1 . In this setting, the sign function is negacyclic. Then, it is

possible to obtain an encryption $\text{ct}' = \text{LWE}_{\mathbf{s}'}^{N,Q}(m')$, $m' = \frac{Q}{2N} f(\frac{q}{t}m + e) + e'$ by proceeding as follows. We construct a polynomial $\mathbf{p} \in \mathcal{R}_Q$ such that $\mathbf{p} = \frac{Q}{2N} \sum_{i=0}^{N-1} X^{N-i-1} \cdot f(i)$ and encode it in a (trivial) RLWE sample $\text{acc} = [\mathbf{0}, \mathbf{p}]$. Then, we apply blind-rotation using acc and ct , yielding $\text{acc}' = \text{RLWE}(X^{\varphi(\text{ct})} \cdot \mathbf{p})$ and finally extract a LWE sample containing the constant term via `SampleExtract`. Looking back at the definition of blind-rotation, we note that $\text{acc}' = \text{RLWE}(\mathbf{p} \cdot X^{\varphi(\text{ct})}) = \text{RLWE}(\underbrace{\mathbf{p} \cdot X^{(q/t) \cdot m + e}}_{=: \mathbf{q}_0})$ and by the construction of $\mathbf{p}$ we have

$$\mathbf{q}_0 = \frac{Q}{2N} \cdot \begin{cases} f((q/t) \cdot m + e) & (q/t) \cdot m + e < N, \\ -f((q/t) \cdot m + e) & (q/t) \cdot m + e \geq N. \end{cases}$$

We note again that the sign-flip occurs as a property of the chosen ring, since over $\mathcal{R}_Q$ it is true that $X^N \equiv -1$. As we required f to be negacyclic, it can be verified that the constant term contains the correct result. Then, obtaining a LWE sample is achieved through use of the `SampleExtract` function. We stress that, although f has $\mathbb{Z}_{2N}$ as domain, the plaintext parameter t must be significantly smaller than $2N$ so that the message m and error e can be distinguished. In practice, f is usually a composition of a rounding function, mapping the phase $\frac{q}{t}m + e$ to the closest multiple of $\frac{q}{t}m$, and the actual function $f' : \mathbb{Z}_t \mapsto \mathbb{Z}_t$ to be evaluated on m . A more intuitive view is that the coefficients of $\mathbf{p}$ are block-wise identical such that ciphertexts encoding the same message but with a different error are mapped to the same function value.

Loosening the restriction on f has been the topic of a significant body of works e.g. [11] [25] [12] [32]. Throughout our work, we circumvent this issue completely by always guaranteeing that for any ct , $\varphi(\text{ct}) < N$. In this setting, any function can be evaluated without needing to care about potential sign-flips.

3.1 Improved Digit Decomposition

In this section, we describe our novel digit-decomposition approach. Recall that we are given an LWE sample $\text{ct} = \text{LWE}_{\mathbf{s}}^{n,q}(m)$ with $q, m \gg 2N$, that is a ciphertext which cannot be decomposed using functional bootstrapping. Furthermore, throughout our exposition we assume that $\mathbf{s} \in \{0, 1\}^n$ as has become common for practical efficiency. Then, we aim to obtain $\text{ct}_i = \text{LWE}_{\mathbf{s}'}^{n',q'}(\tilde{m}_i)$, $\tilde{m}_i = \frac{q'}{\alpha} \cdot m_i + e_i$ for output parameters $q', n', \mathbf{s}'$, such that $m = \sum_i \alpha^i m_i$. We note that the m_i correspond to the digits of m in base α and for ct we do not distinguish between the message and error as our method is independent of the input encoding. Our first step is to express the ciphertext ct itself in base α and obtain $\text{ct}'_i = [\mathbf{a}'_i, b'_i]$ with

$$\text{ct} = \sum_i \alpha^i \text{ct}'_i \pmod{q} \quad (1)$$

and $\|\mathbf{a}'_i\|_\infty \leq \alpha, |b'_i| \leq \alpha$. We stress that the "messages" encoded in ct'_i do *not* correspond to the target digit values and may be seen as random, but fixed. Furthermore, letting $\|\mathbf{s}\|_0 = \beta$ we observe that over $\mathbb{Z}$

$$-\alpha\beta \leq \varphi(\text{ct}'_i) \leq \alpha \Leftrightarrow -\alpha\beta \leq b'_i - \langle \mathbf{a}'_i, \mathbf{s} \rangle \leq \alpha \Leftrightarrow -\alpha\beta \leq b'_i - \sum_{i:\mathbf{s}_i \neq 0} \mathbf{a}'_i \leq \alpha.$$

The derivation highlights that $\varphi(\text{ct}'_i)$ lies in an interval of size $\alpha\beta - (-\alpha) = \alpha(\beta + 1)$. Then, assuming $\alpha(\beta + 1) < N$, we may use ct'_i in a blind-rotation without needing to deal with a sign-flip. We shall perturb each ct'_i by $\alpha\beta$ so that $\varphi(\text{ct}'_i) \in [0, \alpha(\beta + 1)) \subset [0, N)$ holds. Simultaneously, we need to subtract β from the values represented by the upper digits as done in Algorithm 1. There, the decomposition is computed from least to most significant digit such that the perturbations do not affect the invariant given in Equation 1. Furthermore, the ct'_i are given mod $2N$ as we use them for blind-rotation at a later stage.

Algorithm 1 Phase-Decomposition: PDecomp

Input: LWE sample $\text{ct} = \text{LWE}_{\mathbf{s}}^{n,q}(m)$

Input: Bound β , with $\|\mathbf{s}\|_0 \leq \beta$

Output: LWE samples $(\text{ct}'_i)_{0 \leq i \leq \lceil \log_\alpha(q) \rceil}$ with $\varphi(\text{ct}) = \sum_i \alpha^i \varphi(\text{ct}'_i)$

```

1:  $P \leftarrow \alpha\beta$ 
2:  $S \leftarrow \beta$ 
3: for  $i \leftarrow 0$  to  $\lceil \log_\alpha(q) \rceil$  do
4:    $\text{ct}'_i \leftarrow [\text{ct}]_\alpha + [\mathbf{0}, P] \pmod{2N}$   $\triangleright$  Perturb by  $\alpha\beta$  to move interval to  $[0, N)$ 
5:    $\text{ct} \leftarrow \lfloor \frac{\text{ct}}{\alpha} \rfloor - [\mathbf{0}, S]$   $\triangleright$  Correct perturbation to maintain invariant
6: return  $(\text{ct}'_i)_{0 \leq i \leq \lceil \log_\alpha(q) \rceil}$ 

```

Following the previous discussion, we may now write $\varphi(\text{ct}'_i) = x_i + \alpha y_i$ with $x_i < \alpha$, $y_i < \beta + 1$ and we rewrite Equation 1:

$$\begin{aligned}
\text{ct} &= \sum_i \alpha^i \text{ct}'_i \pmod{q} \implies \varphi(\text{ct}) = \sum_i \alpha^i \varphi(\text{ct}'_i) \\
&\implies \varphi(\text{ct}) = \sum_i \alpha^i (x_i + \alpha y_i) = \sum_i \alpha^i x_i + \alpha \sum_i \alpha^i y_i.
\end{aligned}$$

Assuming that $\alpha = \beta + 1$, the previous identity shows that digit decomposition can be reduced to the addition of two numbers whose digits are given by $x_0 x_1 \dots x_k$ and $0 y_0 y_1 \dots y_{k-1}$ for $k = \lceil \log_\alpha(q) \rceil$. In general, it will not be true that $\alpha = \beta + 1$, however we make another observation, namely

$$\begin{aligned}
\left\lfloor \frac{\varphi(\text{ct})}{\alpha^j} \right\rfloor &= \left\lfloor \frac{\sum_i \alpha^i x_i + \alpha \sum_i \alpha^i y_i}{\alpha^j} \right\rfloor \\
&= \left\lfloor \frac{\sum_{i=0}^{j-1} \alpha^i x_i + \alpha \sum_{i=0}^{j-2} \alpha^i y_i}{\alpha^j} \right\rfloor + \sum_{i=j}^k \alpha^{i-j} x_i + \sum_{i=j-1}^k \alpha^{i-j+1} y_i \\
\implies \left\lfloor \frac{\varphi(\text{ct})}{\alpha^j} \right\rfloor &= \left\lfloor \frac{\sum_{i=0}^{j-1} \alpha^i x_i + \alpha \sum_{i=0}^{j-2} \alpha^i y_i}{\alpha^j} \right\rfloor + x_j + y_{j-1} \pmod{\alpha} \\
\implies m_j &= \left\lfloor \frac{\sum_{i=0}^{j-1} \alpha^i x_i + \alpha \sum_{i=0}^{j-2} \alpha^i y_i}{\alpha^j} \right\rfloor + x_j + y_{j-1} \pmod{\alpha}
\end{aligned}$$

The prior derivation now leads us to digit decomposition. Ideally, we would like to compute $\text{acc} = \text{RLWE}(X^{\lfloor \varphi(\text{ct})/\alpha^j \rfloor}) = \text{RLWE}(X^{m_j})$ directly, however, as we assume that $q \gg 2N$ we

cannot use blind-rotation directly on ct . Instead, we give an approach to obtain every digit iteratively using the following recurrence relation(s)

$$\begin{aligned} y'_0 &= 0, \quad y'_{j+1} = \left\lfloor \frac{x'_j}{\alpha} \right\rfloor, \\ x'_j &= y'_j + x_j + \alpha y_j \stackrel{!}{=} y'_j + \varphi(\text{ct}'_j). \end{aligned}$$

For which it holds that $x'_j \pmod{\alpha} = m_j$. Then, we may iteratively construct the sequence $\text{acc}_j = \text{RLWE}(X^{x'_j})$ and extract the digits by multiplying acc_j by an appropriate polynomial such that the digit lies in the constant term of the result. In practice, this step is performed similarly to the function evaluation discussed at the beginning of the section.

For further intuition regarding the method, recall that our target digits are the digits of the sum of $x_0x_1\dots x_k$ and $0y_0y_1\dots y_{k-1}$ in the case $\alpha = \beta + 1$ and that an adder would combine digits x_j and y_{j-1} . Then, the y'_j correspond to the sum of an "input" carry and the digit y_{j-1} , which are added to the digit x_j through computation of x'_j by performing blind-rotation using $\varphi(\text{ct}'_j)$. However, even for the case $\alpha \neq \beta + 1$ the recurrence relations remains true, though we need to proceed carefully. We previously noted that $\varphi(\text{ct}'_j) \in [0, \alpha(\beta+1)) \subset [0, N)$ but it is in general not guaranteed that $x'_j < N$. Before finally stating our algorithms, we derive conditions under which we are sure that the previous inequalities hold. Without loss of generality, we assume that $y'_j \leq \beta + c$ for some constant c . Then, requiring that the next $y'_{j+1} \leq \beta + c$ leads to a bound $c \geq \frac{\beta+1}{\alpha-1} + 1$ and ultimately to the bound $0 \leq x'_j \leq \beta(\alpha+2) + 2$. Finally, requiring $\beta(\alpha+2) + 2 < N$ ensures the correctness of our algorithms.

We give the complete digit decomposition algorithm in Algorithm 2 and state its correctness in Theorem 1. Finally, we note that Algorithm 2 outputs the digits modulo Q with dimension N , which can easily be changed as needed via key- and modulus-switching. The digit decomposition relies on a floored division in the exponent step for which we give a possible algorithm in Algorithm 3 using Algorithm 4 for LWE to RLWE conversion, though note it is not the only way (c.f. [31]).

Algorithm 2 Digit-Decomposition DigitDec

Input: LWE sample $\text{ct} = \text{LWE}_s^{n,q}(m')$

Output: LWE samples $(\text{ct}_{\text{out},j})_{0 \leq j < \lceil \log_\alpha(q) \rceil}$ such that $\varphi(\text{ct}_{\text{out},j}) = \frac{q'}{\alpha} m_j + e_i$ and $m' = \sum_j \alpha^j m_j$,
with $q' = 2N, j = 0$ and $q' = Q$ otherwise.

```

1:  $(\text{ct}_j^p) \leftarrow \text{PhaseDecomp}(\text{ct}, \alpha)$ 
2:  $\text{ct}_{\text{out},0} \leftarrow \text{ct}_0^p \cdot \frac{2N}{\alpha}$ 
3:  $\text{acc} \leftarrow [0, \frac{Q}{\alpha}]$ 
4:  $\mathbf{p} \leftarrow \sum_{i=0}^{N-1} X^{N-i-1} \cdot [i]_\alpha$ 
5:  $\text{acc} \leftarrow \text{BlindRotate}(\text{acc}, \text{ct}_0^p)$   $\triangleright \text{acc} = \text{RLWE}(X^{x'_0})$ 
6: for  $j \leftarrow 1$  to  $\lceil \log_\alpha(q) \rceil$  do
7:    $\text{acc} \leftarrow \text{HomTrunc}(\text{acc}, \alpha)$   $\triangleright \text{acc} = \text{RLWE}(X^{y'_j})$ 
8:    $\text{acc} \leftarrow \text{BlindRotate}(\text{acc}, \text{ct}_j^p)$   $\triangleright \text{acc} = \text{RLWE}(X^{x'_{j+1}})$ 
9:    $\text{acc}_j \leftarrow \text{acc} \cdot \mathbf{p}$   $\triangleright \text{acc}_j = \text{RLWE}(\frac{Q}{\alpha} \cdot m_j + X\tau)$ 
10:  $\text{ct}_{\text{out},j} \leftarrow \text{SampleExtract}(\text{acc}_j, 0)$ 
11: return  $(\text{ct}_{\text{out},j})_{0 \leq j < \lceil \log_\alpha(q) \rceil}$ 

```

Theorem 1. *Correctness of Algorithm 2* Let $\mathcal{B}_{\text{br}}$, $\mathcal{B}_{\text{LWE2RLWE}}$ be the variance induced by blind-rotation and LWE to RLWE switching. Furthermore, let $\text{ct} = \text{LWE}_{\mathbf{s}}^{n,q}(\tilde{m})$ be a LWE sample under $\mathbf{s} \in \{0,1\}^n$ with $\|\mathbf{s}\|_0 \leq \beta$. Then, if $\beta(\alpha + 2) + 2 < N$, Algorithm 2 outputs $(\text{ct}_{\text{out},j})_{0 \leq j \leq \lceil \log_{\alpha}(q) \rceil}$ with $\text{ct}_{\text{out},j} = \text{LWE}_{\mathbf{s}}^{n',q'}(\tilde{m}_j)$, $\tilde{m}_j = \frac{q'}{\alpha} m_j + e_j$, $m_j \leq \alpha$ such that $\varphi(\text{ct}) = \sum_j \alpha^j m_j$ and with variance $\mathcal{B}_{\text{acc},j} \leq \frac{N\alpha^{j+2}}{6} (2\mathcal{B}_{\text{br}} + \frac{N}{\alpha} \mathcal{B}_{\text{LWE2RLWE}})$

Proof. As correctness has been established throughout the section, we focus on the noise bound. Then, let $\mathcal{B}_j$ denote that variance of acc in iteration $0 \leq j \leq \log_{\alpha}(q)$ and observe that $\mathcal{B}_{j+1} \leq \alpha \mathcal{B}_j + \frac{N}{\alpha} \mathcal{B}_{\text{LWE2RLWE}} + \mathcal{B}_{\text{br}}$ or in closed form $\mathcal{B}_{j+1} \leq \alpha^j (2\mathcal{B}_{\text{br}} + \frac{N}{\alpha} \mathcal{B}_{\text{LWE2RLWE}})$ as $\mathcal{B}_0 = \mathcal{B}_{\text{br}}$, and finally $\mathcal{B}_{\text{acc},j} \leq \frac{N\alpha^{j+2}}{6} (2\mathcal{B}_{\text{br}} + \frac{N}{\alpha} \mathcal{B}_{\text{LWE2RLWE}})$ from the fact that $\|\mathbf{p}\|_0 \leq N$ and $\|\mathbf{p}\|_{\infty} \leq \alpha$. $\square$

Algorithm 3 Homomorphic Truncation: HomTrunc

Input: RLWE sample $\text{acc} = \text{RLWE}(\frac{Q}{t} X^m)$

Input: Truncation factor γ

Output: RLWE sample $\text{acc}_{\text{out}} = \text{RLWE}(\frac{Q}{t} \cdot X^{\lfloor \frac{m}{\gamma} \rfloor})$

```

1: for  $i = 0$  to  $i = \frac{N}{\gamma} - 1$  do
2:    $\text{ct}_i^+ \leftarrow [0, 0]$ 
3:   for  $j = 0$  to  $j = \gamma - 1$  do
4:      $\text{ct}_j \leftarrow \text{SampleExtract}(\text{acc}, \gamma \cdot i + j)$ 
5:      $\text{ct}_i^+ \leftarrow \text{ct}_i^+ + \text{ct}_j$ 
6:    $\text{acc}_i \leftarrow \text{LWE2RLWE}(\text{ct}_i^+)$ 
7:  $\text{acc}_{\text{out}} \leftarrow \sum_{i=0}^{\frac{N}{\gamma}-1} \text{acc}_i \cdot X^i$ 
8: return  $\text{acc}_{\text{out}}$ 

```

Theorem 2. *Correctness of Algorithm 3* Let $\mathcal{B}_{\text{LWE2RLWE}}$, be the variance induced LWE to RLWE conversion and let $\text{acc} = \text{RLWE}(\frac{Q}{t} X^m)$ be a RLWE sample with variance bound $\mathcal{B}_{\text{acc}}$ and plaintext space parameter t . Then, Algorithm 3 outputs a RLWE sample $\text{acc}_{\text{out}} = \text{RLWE}(\frac{Q}{t} X^{\lfloor \frac{m}{\gamma} \rfloor})$ with variance $\mathcal{B} \leq \gamma \cdot \mathcal{B}_{\text{acc}} + \frac{N}{\gamma} \cdot \mathcal{B}_{\text{LWE2RLWE}}$

Proof. First, note that sample-extraction induces no noise growth. Then, ct_i^+ has a variance of $\gamma \cdot \mathcal{B}_{\text{acc}}$ and therefore acc_i has a variance of $\gamma \cdot \mathcal{B}_{\text{acc}} + \mathcal{B}_{\text{LWE2RLWE}}$ in the constant coefficient, and variance of $\mathcal{B}_{\text{LWE2RLWE}}$ in the remaining coefficients. Finally, as acc_{out} shifts each acc_i by X^i , the constant coefficients do not overlap and the bound follows. With regards to the correctness of the approach, we note that the encoded message in acc is a monomial. Therefore, there is a single ct_i^+ that contains the scaled term $\frac{Q}{t}$, namely the one where $i = \lfloor \frac{m}{\gamma} \rfloor$. Then, since each acc_i is shifted by X^i it will hold that $\text{acc}_{\text{out}} = \text{RLWE}(q), q_i = \frac{Q}{t}$ iff $i \neq \lfloor \frac{m}{\gamma} \rfloor$. $\square$

Algorithm 4 LWE to RLWE conversion LWE2RLWE

Input: LWE sample $\text{ct} = [\mathbf{a}, b] = \text{LWE}_{\mathbf{s}}^{Q,N}(m)$
Output: RLWE sample $\text{acc} = \text{RLWE}_{\mathbf{s}}(m)$ with $\mathbf{s}_i = \mathbf{s}_i$
1: $\text{acc} \leftarrow N^{-1} \cdot [\mathbf{a}, b] \pmod{Q}$ with $\mathbf{a}_0 = \mathbf{a}_0$, $\mathbf{a}_i = -\mathbf{a}_{N-i}$
2: **for** $i = 0$ to $i = \log_2(N)$ **do**
3: $\text{acc} \leftarrow \text{acc} + \text{Aut}_{\sigma_{2^i+1}}(\text{acc})$
4: **return** acc

Theorem 3. *Correctness of Algorithm 4* Let $\text{ct} = \text{LWE}_{\mathbf{s}}^{Q,N}(m)$ and let $\mathcal{B}_{\text{auto}}$ be a bound on the variance induced by automorphism evaluation. Then, Algorithm 4 outputs a RLWE sample $\text{acc} = \text{RLWE}(m)$ encoding m in the constant term and with an output variance of $\mathcal{B} \leq \mathcal{B}_{\text{acc}} + N^2 \mathcal{B}_{\text{auto}}$

Proof. We refer to [6]. □

Extensions To close this subsection, we give a range of approaches that extend our digit-decomposition method. We omit the details as they follow quickly.

Restarting The output variance of Algorithm 2 grows proportionally to the digit index i and the term α^{i+2} may become as large as the input modulus q . The practical impact depends strongly on the relationship between q and Q and concrete values for $\mathcal{B}_{\text{br}}, \mathcal{B}_{\text{LWE2RLWE}}$. However, it is possible at any time to "restart" the algorithm in a given iteration as the carry y'_j can be extracted just like the digit and via bootstrapping it becomes possible to construct a new acc encoding the same values state as before but with lower noise.

Digit-Homomorphic Function Evaluation Note that in Algorithm 2 we encoded the function $f : x \mapsto [x]_{\alpha}$ to perform the decomposition. However, nothing is stopping us from applying a different lookup table and in fact any function can be applied at this stage. Assume that for a given function $f : \mathbb{Z}_q \mapsto \mathbb{Z}_q$ there exist a set of functions $f_i : \mathbb{Z}_{\alpha} \mapsto \mathbb{Z}_q$ such that $f(m) = \sum_i f_i(m_i)$. Then f can be easily evaluated by adapting the scaling parameter for acc . Possible functions include certain Lipschitz or in some cases by relying on a constructive version of the Kolmogorov Arnold Representation theorem [26].

Residue Number System (RNS) Decomposition Our goal was to obtain a digit decomposition. However, we note that it is straightforward to extend the previous algorithms to a composite plaintext modulus, such that the digits correspond to RNS digits instead of Radix- α digits, assuming the RNS limbs correspond to small moduli or powers thereof. Conceptually, such ideas have been leveraged in the past, see e.g. [3].

Probabilistic Digit Decomposition Initially, bounds on the phase of the digits obtained by Algorithm 1 assumed the worst case. In practice, one can view the coefficients of the digit ciphertexts to be uniformly distributed over $[0, \alpha) \cap \mathbb{Z}$. Hence, the phase of LWE sample $\text{ct}' = [\mathbf{a}'_i, b'_i]$, $b' - \sum_{i: \mathbf{s}_i \neq 0} \mathbf{a}'_i s_i$ can be treated as a sample of (scaled) Irwin-Hall distribution with parameter $\beta + 1$. Then, instead of assuming the worst case, we may instead determine α, β such that $\varphi(\text{ct})$ is bounded in absolute value by N with large probability (instead of probability equal to 1), e.g., $p_{\text{fail}} \leq 2^{-60}$, as is commonly done in the context of fully homomorphic encryption, which may lead to some performance gains for HomTrunc.

3.2 Full Ring Bootstrapping

We discuss a major application of the digit and phase splitting approach, namely the task of evaluating a negacyclic LUT $f : \mathbb{Z}_t \mapsto \mathbb{Z}_t$. In the normal setting, such evaluations are restricted to plaintext spaces $\mathbb{Z}_t$ with $t \ll N$ since the message¹ is encoded $\varphi(\text{ct}) = \alpha m + e \pmod{q}$ and the scaling parameter $\alpha = \frac{q}{t}$ separates the message from the error e enabling us to correctly decode m . While effective, it puts strong bounds on t unless we accept the application of costly methods such as [20]. We now show how to use our digit decomposition to significantly raise the plaintext space to $t = 2N$.

First, we specifically assume that the modulus of the LWE sample to be bootstrapped should be equal to $q = \alpha 2N$. Then, as previously for the digit decomposition, we can define ciphertexts $\text{ct}_{\text{lo}} = [\text{ct}]_{\alpha} + \alpha\beta \pmod{2N}$ and $\text{ct}_{\text{hi}} = \lfloor \frac{\text{ct}}{\alpha} \rfloor - \beta \pmod{2N}$.

We shall not decompose ct_{hi} further and note that $\varphi(\text{ct}_{\text{hi}}) = m + y'$ where y' is exactly the overflow y'_1 as used in the digit decomposition. Hence, we can easily correct this error and obtain a RLWE(X^m) by relying on truncation and ct_{lo} . We give the specific steps below in Algorithm 5 and state its correctness in Theorem 4.

Algorithm 5 Full-Ring Bootstrap FrBoot

Input: LWE sample $\text{ct} = \text{LWE}_s^{n,q}(m')$ with $m' = \alpha m + e = \frac{q}{2N}m + e$

Input: Function/Lookup Table $f : \mathbb{Z}_{2N} \mapsto \mathbb{Z}_{2N}$

Output: LWE sample $\text{ct}_{\text{out}} = \text{LWE}_s^{N,Q}(m')$ with $m' = \frac{Q}{2N}f(m) + e'$

- 1: $\text{ct}_{\text{lo}} \leftarrow [\text{ct}]_{\alpha} + \alpha\beta$
 - 2: $\text{ct}_{\text{hi}} \leftarrow \lfloor \frac{\text{ct}}{\alpha} \rfloor - \beta$
 - 3: $\text{acc} \leftarrow [0, \frac{Q}{2N}]$
 - 4: $\mathbf{p} \leftarrow \sum_{i=0}^{N-1} X^{N-i-1} f(i)$
 - 5: $\text{acc} \leftarrow \text{BlindRotate}(\text{acc}, \text{ct}_{\text{lo}})$
 - 6: $\text{acc} \leftarrow \text{HomTrunc}(\text{acc}, \alpha)$
 - 7: $\text{acc} \leftarrow \text{acc} \cdot \mathbf{p}$
 - 8: $\text{acc} \leftarrow \text{BlindRotate}(\text{acc}, \text{ct}_{\text{hi}})$
 - 9: $\text{ct}_{\text{out}} \leftarrow \text{SampleExtract}(\text{acc}_i, 0)$
 - 10: **return** ct_{out}
-

Theorem 4. *Correctness of Algorithm 5* Let $\mathcal{B}_{\text{br}}$, $\mathcal{B}_{\text{LWE2RLWE}}$ be the variance induced by blind-rotation and LWE to RLWE conversion respectively. Furthermore, let $\text{ct} = \text{LWE}_s^{n,q}(\tilde{m})$, $\tilde{m} = \alpha m + e$ be a LWE sample and specifically $\alpha = \frac{q}{2N}$, and $f : \mathbb{Z}_{2N} \mapsto \mathbb{Z}_{2N}$ be a negacyclic function that is $f(x + N) = -f(x)$. Then, if $\|\mathbf{s}\|_0 \leq \beta$ and $\beta(\alpha + 2) + 2 < N$, Algorithm 5 outputs a LWE sample $\text{ct}_{\text{out}} = \text{LWE}_s^{N,Q}(m')$ with $m' = \frac{Q}{2N}f(m) + e'$ such that the error variance $\mathcal{B}_{e'}$ such that $\mathcal{B}_{e'} \leq \frac{N^3}{6} \left(\alpha \cdot \mathcal{B}_{\text{br}} + \frac{N}{\alpha} \mathcal{B}_{\text{LWE2RLWE}} \right) + \mathcal{B}_{\text{br}}$.

Proof. In general, correctness follows from Theorem 1 but we note that no further truncation on ct_{hi} is necessary as the component of the phase that might overflow, will also overflow modulo q , i.e. it behaves as we expect for a negacyclic function and the function is evaluated equivalently to prior literature e.g. [11] [25]. In terms of noise variance we note that in

¹ E.g. $t = 16, N = 2^{11}$ are common parameters

Line 6, the noise variance of acc is bounded by $\alpha \mathcal{B}_{\text{br}} + \frac{N}{\alpha} \mathcal{B}_{\text{LWE2RLWE}}$. Since $\|\mathbf{p}\|_\infty \leq N$ and $\|\mathbf{p}\|_\infty \leq N$ the variance follows.

Although Algorithm 5 is a direct consequence of our digit decomposition approach, the fact remains that the target output variance remains large and $O(N^4)$ overall as $\mathcal{B}_{\text{br}}, \mathcal{B}_{\text{LWE2RLWE}} \in O(N)$ and $\|\mathbf{p}\|_\infty < N$. Before describing an approach to correct this large noise growth, we show how to extent the method to evaluate multiple functions without additional blind-rotations in Algorithm 6 and give its correctness criteria in Theorem 5.

Algorithm 6 Multi Value Full-Ring Bootstrap MvFrBoot

Input: LWE sample $\text{ct} = \text{LWE}_s^{n,q}(m)$ with $m = \alpha m + e = \frac{q}{2N}m + e$

Input: Functions/Lookup Tables $f_i : \mathbb{Z}_{2N} \mapsto \mathbb{Z}_{2N}, i \leq k$

Output: LWE samples $\text{ct}_{\text{out},i} = \text{LWE}_s^{N,Q}(m'_i)$ with $m'_i = \frac{Q}{2N}f_i(m) + e'$

```

1:  $\text{ct}_{\text{lo}} \leftarrow \lfloor \text{ct} \rfloor_\alpha + \alpha \frac{\beta+1}{2}$ 
2:  $\text{ct}_{\text{hi}} \leftarrow \lfloor \frac{\text{ct}}{\alpha} \rfloor - \frac{\beta+1}{2}$ 
3:  $\text{acc} \leftarrow [\mathbf{0}, \frac{Q}{2N}]$ 
4:  $\text{acc} \leftarrow \text{BlindRotate}(\text{acc}, \text{ct}_{\text{lo}})$ 
5:  $\text{acc} \leftarrow \text{HomTrunc}(\text{acc}, \alpha)$ 
6:  $\text{acc} \leftarrow \text{BlindRotate}(\text{acc}, \text{ct}_{\text{hi}})$ 
7: for  $i \leftarrow 0$  to  $k-1$  do
8:    $\mathbf{p}^{(i)} \leftarrow \sum_{j=0}^{N-1} X^{N-j-1} f_i(j)$ 
9:    $\text{acc}_i \leftarrow \text{acc} \cdot \mathbf{p}^{(i)}$ 
10:   $\text{ct}_{\text{out},i} \leftarrow \text{SampleExtract}(\text{acc}_i, 0)$ 
11: return  $\text{ct}_{\text{out},i}$ 

```

Theorem 5. *Correctness of Algorithm 6* Let $\mathcal{B}_{\text{br}}, \mathcal{B}_{\text{LWE2RLWE}}$ be the variance induced by blind-rotation and homomorphic truncation respectively. Furthermore, let $\text{ct} = \text{LWE}_s^{n,q}(\tilde{m})$, $\tilde{m} = \alpha m + e$ be a LWE sample and specifically $\alpha = \frac{q}{2N}$, and $f_i : \mathbb{Z}_{2N} \mapsto \mathbb{Z}_{2N}, 0 \leq i < k$ be a set of negacyclic functions that is $f_i(x + N) = -f_i(x)$. Then, if $\|\mathbf{s}\|_0 \leq \beta$ and $\beta(\alpha + 2) + 2 < N$, Algorithm 5 outputs a set of LWE samples $\text{ct}_{\text{out},i} = \text{LWE}_s^{N,Q}(m'_i)$ with $m'_i = \frac{Q}{2N}f_i(m) + e'_i$ such that the output variance $\mathcal{B}_{e'}$ is bounded as $\mathcal{B}_{e'} \leq ((\alpha + 1) \cdot \mathcal{B}_{\text{br}} + \frac{N}{\alpha} \mathcal{B}_{\text{LWE2RLWE}}) \cdot \frac{N^3}{6}$

Proof. We note that in Line 6 we have that $\text{acc} = \text{RLWE}(X^m)$ and correctness follows from the properties of the ring $\mathcal{R}_Q$ and negacyclicity of f_i . The output variance follows as before. $\square$

As touched upon before, the noise growth of the functional bootstrapping methods grows quickly with the ring dimension. We describe how to circumvent this issue in Algorithm 7. The intuition behind this approach is equivalent to that of gadget-decomposition used for RGSW samples [19] in that we construct multiple "digit" RLWE sample w.r.t to a basis L , such that the LUT polynomial(s) can be decomposed into such with coefficients bounded by L .

Theorem 6. *Correctness of Algorithm 7* Let $\mathcal{B}_{\text{br}}, \mathcal{B}_{\text{LWE2RLWE}}$ be the variance induced by blind-rotation and homomorphic truncation respectively. Furthermore, let $\text{ct} = \text{LWE}_s^{n,q}(\tilde{m})$, $\tilde{m} =$

Algorithm 7 Low-noise Multi Value Full-Ring Bootstrap MVFrBoot**Input:** LWE sample $\text{ct} = \text{LWE}_s^{n,q}(m)$ with $m = \alpha m + e = \frac{q}{2N}m + e$ **Input:** Functions/Lookup Tables $f_i : \mathbb{Z}_{2N} \mapsto \mathbb{Z}_{2N}, i \leq k$ **Input:** Basis L **Output:** LWE samples $\text{ct}_{\text{out},i} = \text{LWE}_s^{N,Q}(m'_i)$ with $m'_i = \frac{Q}{2N}f_i(m) + e'$

```

1:  $\text{ct}_{\text{lo}} \leftarrow \lfloor \text{ct} \rfloor_\alpha + \alpha \frac{\beta+1}{2}$ 
2:  $\text{ct}_{\text{hi}} \leftarrow \lfloor \frac{\text{ct}}{\alpha} \rfloor - \frac{\beta+1}{2}$ 
3: for  $j = 0$  to  $j = \log_L(2N) - 1$  do
4:    $\text{acc}_j \leftarrow [\mathbf{0}, L^j \cdot \frac{Q}{2N}]$ 
5:    $\text{acc}_j \leftarrow \text{BlindRotate}(\text{acc}_j, \text{ct}_{\text{lo}})$ 
6:    $\text{acc}_j \leftarrow \text{HomTrunc}(\text{acc}_j, \alpha)$ 
7:    $\text{acc}_j \leftarrow \text{BlindRotate}(\text{acc}_j, \text{ct}_{\text{hi}})$ 
8: for  $i \leftarrow 0$  to  $k - 1$  do
9:    $\mathbf{p}^{(i)} \leftarrow \sum_{j=0}^{N-1} X^{N-j-1} f_i(j)$ 
10:  Write  $\mathbf{p}^{(i)} \leftarrow \sum_j L^j \mathbf{p}^{(i,j)}$ 
11:   $\text{acc}' \leftarrow \sum_j \text{acc}_j \cdot \mathbf{p}^{(i,j)}$ 
12:   $\text{ct}_i^Q \leftarrow \text{SampleExtract}(\text{acc}', 0)$ 
13: return  $\text{ct}_{\text{out}}$ 

```

$\alpha m + e$ be a LWE sample and specifically $\alpha = \frac{q}{2N}$, and $f_i : \mathbb{Z}_{2N} \mapsto \mathbb{Z}_{2N}, 0 \leq i < k$ be a set of negacyclic functions that is $f_i(x + N) = -f_i(x)$. Then, if $\|\mathbf{s}\|_0 \leq \beta$ and $\beta(\alpha + 2) + 2 < N$, Algorithm 5 outputs a set of LWE samples $\text{ct}_{\text{out},i} = \text{LWE}_s^{N,Q}(m'_i)$ with $m'_i = \frac{Q}{2N}f_i(m) + e'_i$ that the output variance $\mathcal{B}_{e'}$ is bounded as $\mathcal{B}_{e'} \leq ((\alpha + 1) \cdot \mathcal{B}_{\text{br}} + \frac{N}{\alpha} \mathcal{B}_{\text{LWE2RLWE}}) \cdot \log_L(2N) \cdot \frac{L^2}{6}$

Proof. Similarly as before, we note that the variance of acc_j is bounded by $((\alpha + 1) \cdot \mathcal{B}_{\text{br}} + \frac{N}{\alpha} \mathcal{B}_{\text{LWE2RLWE}})$. Furthermore, note that $\|\mathbf{p}^{(i,j)}\|_\infty \leq L$ and the variance bound follows. For correctness, we note that

$$\begin{aligned}
\text{acc}' &= \sum_j \text{RLWE} \left(\frac{Q}{2N} \cdot L^j \cdot X^m \right) \cdot \mathbf{p}^{(i,j)} = \sum_j \text{RLWE} \left(\frac{Q}{2N} \cdot L^j \cdot \mathbf{p}^{(i,j)} \cdot X^m \right) \\
&= \sum_j \text{RLWE} \left(\frac{Q}{2N} \cdot \mathbf{p}^{(i)} \cdot X^m \right)
\end{aligned}$$

where the final step followed by definition of the $\mathbf{p}^{(i,j)}$. □

To close this subsection, we explain how to evaluate arbitrary function. Note that all our methods have been restricted to the evaluation of negacyclic LUTs. All previous works to remove this restriction can be immediately applied here. Hence, we do not discuss them further and refer the reader to [25]. However, we also observe that in the small plaintext space regime we have *already* constructed a full domain approach, though this was not our intention. Going back to Section 3.1, we know that as long as $\alpha(\beta + 2) + 2 < N$, $\alpha = \frac{q}{t}$, no overflow shall occur and in fact Algorithm 5 is a full domain approach.

4 Implementation and Evaluation

In this section we give a practical evaluation and theoretical comparison to related works

4.1 Theoretical Comparison

In this section we give a theoretical comparison to related works. We focus on our digit-decomposition and shall not compare our full-ring functional bootstrapping methods since it is not the main contribution, though note that similar properties have been obtained in [31]. We give a concise overview of the differences in Table 1.

Table 1: Overview of comparison to [28] with LWE modulus q , ring dimension & modulus N, Q and α corresponds to decomposition basis, β to the hamming weight of the binary secret key

Work	#Blind-Rotations	Conditions	Output Variance	q_{out}
[28]	$1 + 2 \lfloor \frac{\log_2(q) - \log_2(2N/\alpha)}{\log_2(2N/\alpha)} \rfloor$	$N > 2\alpha\beta, \frac{N}{\alpha} \geq 4\beta$	$2 \mathcal{B}_{\text{br}}$	$2N$
Ours	$1 + \lfloor \frac{\log_2(q)}{\log_2(\alpha)} \rfloor$	$\beta(\alpha + 2) + 2 < N$	$\frac{N\alpha^{i+2}}{6} (2 \mathcal{B}_{\text{br}} + \frac{N}{\alpha} \mathcal{B}_{\text{LWE2RLWE}})$	Q

We compare our digit decomposition against [28] which to the best of our knowledge corresponds to the state of the art. First, we stress than in [28], α_{LMP} denotes a scaling factor and **not** the radix and the output basis is $\alpha = \frac{2N}{\alpha_{LMP}}$. Furthermore, β acts as a bound on $\|\mathbf{s}\|_1$ and a bound on the modulus switching error $\beta' \geq \frac{\|\mathbf{s}\|_1 + 1}{3}$. We observe that for $\mathbf{s} \in \{0, 1\}$ $\|\mathbf{s}\|_0 = \|\mathbf{s}\|_1$ and therefore only the first bound remains useful. Then, according to Theorem 2 in [28] it must hold that

$$\alpha_{LMP} \geq 4\beta \implies \frac{2N}{\alpha} \geq 4\beta \implies N \geq 2\alpha\beta.$$

Recall that our Algorithm 2 assumes that $\beta(\alpha + 2) + 2 < N$. Based on the previous inequality we are tempted to claim that our approach is easier to leverage as the conditions for [28] imply ours for any $\alpha \geq 2$, but in practice it has little effect on parameters. However, β also acts as a bound on the input error in [28] and it is assumed that the scaling parameter $\alpha_{LMP} > 4\beta$ in order to guarantee correctness. This bound is strictly necessary, as intermediate digits are re-used to modify the input in an appropriate way to obtain the next digit. However, we make no such assumption and Algorithm 2 always returns the digits of the input phase. In general, our methodology does not treat message and error separately and may have further applications as similar ideas have been leveraged effectively in approximate FHE schemes such as CKKS. Furthermore, as the digit decomposition from [28] iteratively modifies the input plaintext it must hold that the input modulus q is smaller than the ring modulus Q , a restriction which is not present in our method.

Next, we focus on performance and continue our analysis noting that [28] requires $1 + 2 \lfloor \frac{\log(q) - \log(\alpha)}{\log(\alpha)} \rfloor$ blind-rotations on average whereas our approach performs $1 + \lfloor \frac{\log_2(q)}{\log_2(\alpha)} \rfloor$ operations in total. For completeness, we mention that the homomorphic truncation step

can be realized efficiently and its impact on performance is negligible by relying on recent works [27]. Although the amount of blind-rotations in our approach is halved, this must not translate into a corresponding speedup immediately. With every digit, the variance of outputs of our digit decomposition algorithm increases, which may require less efficient parameters. On the other hand, this can be compensated, as mentioned in Section 3, by restarting after a few iterations. Furthermore, we argue that the larger output variance is not detrimental to our method as we can obtain low-variance digits if desired. In this case, we would use a scaling parameter corresponding to $\frac{2\alpha Q}{q}$ instead of $\frac{\alpha Q}{q}$. By construction our digits are smaller than α and accordingly encoding such values with the aforementioned scale leaves the most significant bit at 0. Then, every digit can be key- and modulus-switched and bootstrapped afterwards. By virtue of the most significant bit being zero, the issue of negacyclicity is resolved and the total number of blind-rotations is the same as [28]. Finally, if such a refreshing step is performed, it may be executed in parallel, still yielding a faster approach than [28].

To close this subsection, we briefly discuss another recent work. In [14], the authors design a novel approach to decompose larger LWE samples into digits, starting from the most significant bits. We note that the approach is fundamentally different from ours and [28]. On the one hand, [14] does not yield an exact decomposition and requires additional bootstrapping operations to make it exact, in which case there is no advantage over [28]. On the other hand, [14] assumes that the most significant bit of the message is known or enforced to be constant. In [14] the authors consider the summation of large precision data as application. Then, to maintain the invariant that the MSB is known, bootstrapping become necessary even when the noise level is low, as the summation of t , k -bit values yields may grow to $t \cdot k$ bits, leading to an overflow into the MSB. No such restriction on the MSB is necessary for [28] and our work. Finally, the comparison in Table 1 [14] is imprecise and claims that the input variance for [28] is propagated to the output digits, which holds only for a single digit, the least significant one, which may not even be part of the encoded message depending on the scaling parameter used. For all other digits, the output variance of [28] is bounded by twice the bootstrapping variance and the least significant digit can be bootstrapped increasing the total cost of the digit decomposition by a constant amount.

4.2 Implementation and Experiments

We implement our approach using a heavily modified version of OpenFHE [1] and Intel Hexl [4]. We note that although the basic digit-decomposition approach of [28] is implemented in OpenFHE, it is not possible to change the decomposition basis. Therefore we chose to re-implement the method from scratch. All experiments were performed on the same machine with an Intel i7 14700KF processor and 64GB of RAM.

We use identical parameters for our comparisons, relying on the justification we obtained in the previous section and give those parameters in Table 2 and Table 3. Furthermore, despite the theoretically higher output variance of Algorithm 2, we note that after key- and modulus-switching to the same output modulus as [28] we may target the same output variance $\mathcal{B}_{\text{out}}^2$ yielding a failure probability of 2^{-60} . For completeness, we point out that in the case of Parameter Set II, we have to restart (c.f. Section 3.1) the algorithm after the fourth digit. We note that q, α, β may be adapted during the evaluation to discuss their impact on performance, but we always require that the hamming weight $\|\mathbf{s}\|_0 = \beta$ yields 128-bit

secure parameters determined with the lattice estimator². Furthermore, we have previously treated blind-rotation as a black-box but perform our evaluation using the known approach in [8] using the optimizations from [5]. For an in-depth discussion of the inner workings on blind-rotations and other employed procedures we refer to [23].

Table 2: Parameter Set I used for out comparison. Q is ring modulus, N the ring dimension, q, n the LWE modulus and dimension respectively. L_{br} and k correspond to the blind-rotation basis and digits respectively. Q_{ksk} is an auxilliary modulus for key-switching and L_{ksk} the basis used. σ^2 is the error variance employed.

Parameter	$ \log_2(Q) $	N	q	n	L_{br}	k	$ \log_2(Q_{ksk}) $	L_{ksk}	σ^2
Value	54	$2^{11} 2^{22}, 2^{32} $	1800	$2^{11} 5 $	42	2^6	3.19^2		

Table 3: Parameter Set II used for out comparison. Q is ring modulus, N the ring dimension, q, n the LWE modulus and dimension respectively. L_{br} and k correspond to the blind-rotation basis and digits respectively. Q_{ksk} is an auxilliary modulus for key-switching and L_{ksk} the basis used. σ^2 is the error variance employed.

Parameter	$ \log_2(Q) $	N	q	n	L_{br}	k	$ \log_2(Q_{ksk}) $	L_{ksk}	σ^2
Value	54	$2^{11} 2^{48} $	3072	$2^{11} 5 $	42	2^6	3.19^2		

We give the obtained performance measurements in Table 4 for $q = 2^{22}$ and in Table 5 for $q = 2^{32}$. A first general observation is that the concrete speedup is highly dependent on the value of α and β . Indeed, as we can rely on the fact the phase of the digits yielded by Algorithm 1 are bounded by $(\alpha + 1)\beta$ c.f. Equation 1, we may select the smallest β that yields (at least) 128-bit secure parameters. This is exemplified in the case $\alpha = 16$ where a value of $\beta = 116$ yields a large security parameter, but the performance gap between Algorithm 2 and [28] is not as large as desired. On the other hand, reducing β to $\beta = 60$ leads to a significantly larger speedup that also translates to the case $\alpha = 32$ or $q = 2^{32}$. Furthermore, we note that a larger modulus leads to bigger speedups which is expected. While our approach is twice as fast asymptotically, the number of actual blind-rotations performed in the chosen settings do not follow this relation as Algorithm 2 requires a few number of additional blind-rotations equal to $1 + \lfloor \log_\alpha(2N) \rfloor$, which is independent of the value of q . However, the impact of additional blind-rotations quickly disappears if we look at Table 6. There both the LWE modulus and dimension are significantly larger than before, causing an increase in the number of digits and cost of blind-rotation. Consequently, our speedup over [28] grows closer to the optimal value and we finally note that for Parameter Set II, it would be possible to reduce the hamming weight further and use an even larger

² <https://github.com/malb/lattice-estimator>

basis α , but we refrained from doing so in order to avoid approaching a regime that could be attacked by recent attacks on sparse-LWE³.

Table 4: Decomposition performance comparison for $q = 2^{22}$, Set I. Time is given in seconds. For a given hamming weight β , we also include the induced security level λ

Approach	Parameters		
	$\alpha = 16, \beta = 60, \lambda = 150$	$\alpha = 16, \beta = 116, \lambda = 210$	$\alpha = 32, \beta = 60, \lambda = 150$
[28]	1.6s	1.6s	1.1s
Algorithm 2	1.2s	1.5s	0.9s
Relative Speedup	$1.33\times$	$1.06\times$	$1.22\times$

Table 5: Decomposition performance comparison for $q = 2^{32}$, Set II. Time is given in seconds. For a given hamming weight β , we also include the induced security level λ

Approach	Parameters		
	$\alpha = 16, \beta = 60, \lambda = 128$	$\alpha = 16, \beta = 116, \lambda = 160$	$\alpha = 32, \beta = 60, \lambda = 128$
[28]	2.7s	2.7s	2.2s
Algorithm 2	1.8s	2.1s	1.4s
Relative Speedup	$1.50\times$	$1.28\times$	$1.57\times$

Table 6: Decomposition performance comparison for $q = 2^{48}$, Set II. Time is given in seconds. For a given hamming weight β , we also include the induced security level λ

Approach	Parameters		
	$\alpha = 16, \beta = 60, \lambda = 300$	$\alpha = 16, \beta = 116, \lambda = 600$	$\alpha = 32, \beta = 60, \lambda = 300$
[28]	8.6s	8.6s	7.5s
Algorithm 2	4.6s	5.3s	3.9s
Relative Speedup	$1.86\times$	$1.62\times$	$1.92\times$

5 Conclusion

In this work we described a new approach to digit decomposition for LWE encrypted messages and which outperforms prior work by up to 92%. Furthermore, we briefly highlighted

³ We refer to [24] for a discussion.

how to leverage the same methodology to significantly extend the plaintext space used in functional bootstrapping. However, we left several questions for future work: first, it is an open question whether it is possible to perform digit decomposition without using (or key-switching to) a sparse key. Furthermore, all digit-decompositions methods cannot be executed in parallel and the latency to obtain the most significant digit is linear in the number of digits. Finally, it is unclear whether our approach can be carried over to other FHE schemes (e.g. CKKS), as has been done for [28].

Acknowledgments. This work was partly supported by the Flemish Government through FWO SBO project MOZAIK S003321N. It was also funded in part by CyberSecurity Research Flanders with reference number VR20192203.

References

1. Al Badawi, A., Bates, J., Bergamaschi, F., Cousins, D.B., Erabelli, S., Genise, N., Halevi, S., Hunt, H., Kim, A., Lee, Y., Liu, Z., Micciancio, D., Quah, I., Polyakov, Y., R.V., S., Rohloff, K., Saylor, J., Suponitsky, D., Triplett, M., Vaikuntanathan, V., Zucca, V.: OpenFHE: Open-source fully homomorphic encryption library. In: Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography. pp. 53–63. WAHC’22, Association for Computing Machinery, New York, NY, USA (2022), <https://doi.org/10.1145/3560827.3563379>
2. Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of learning with errors. *J. Math. Cryptol.* **9**(3), 169–203 (2015), <http://www.degruyter.com/view/j/jmc.2015.9.issue-3/jmc-2015-0016/jmc-2015-0016.xml>
3. Bernard, O., Joye, M.: Approximate CRT-based gadget decomposition for fully homomorphic encryption. In: Cid, C., Yanai, N. (eds.) *Advances in Information and Computer Security*. pp. 45–64. Springer Nature Singapore, Singapore (2026)
4. Boemer, F., Kim, S., Seifu, G., de Souza, F.D., Gopal, V., et al.: Intel HEXL (release 1.2). <https://github.com/intel/hexl> (2021)
5. Bourse, F., Minelli, M., Minihold, M., Paillier, P.: Fast homomorphic evaluation of deep discretized neural networks. In: Shacham, H., Boldyreva, A. (eds.) *CRYPTO 2018, Part III*. LNCS, vol. 10993, pp. 483–512. Springer, Cham (Aug 2018)
6. Chen, H., Dai, W., Kim, M., Song, Y.: Efficient homomorphic conversion between (ring) LWE ciphertexts. In: Sako, K., Tapp, N.O. (eds.) *ACNS 21 International Conference on Applied Cryptography and Network Security, Part I*. LNCS, vol. 12726, pp. 460–479. Springer, Cham (Jun 2021)
7. Cheon, J.H., Kim, A., Kim, M., Song, Y.S.: Homomorphic encryption for arithmetic of approximate numbers. In: Takagi, T., Peyrin, T. (eds.) *ASIACRYPT 2017, Part I*. LNCS, vol. 10624, pp. 409–437. Springer, Cham (Dec 2017)
8. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In: Cheon, J.H., Takagi, T. (eds.) *ASIACRYPT 2016, Part I*. LNCS, vol. 10031, pp. 3–33. Springer, Berlin, Heidelberg (Dec 2016)
9. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster packed homomorphic operations and efficient circuit bootstrapping for TFHE. In: Takagi, T., Peyrin, T. (eds.) *ASIACRYPT 2017, Part I*. LNCS, vol. 10624, pp. 377–408. Springer, Cham (Dec 2017)
10. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology* **33**(1), 34–91 (Jan 2020)
11. Chillotti, I., Ligier, D., Orfila, J.B., Tap, S.: Improved programmable bootstrapping with larger precision and efficient arithmetic circuits for TFHE. In: Tibouchi, M., Wang, H. (eds.) *ASIACRYPT 2021, Part III*. LNCS, vol. 13092, pp. 670–699. Springer, Cham (Dec 2021)

12. Clet, P.E., Boudguiga, A., Sirdey, R., Zuber, M.: ComBo: A novel functional bootstrapping method for efficient evaluation of nonlinear functions in the encrypted domain. In: El Mraabet, N., De Feo, L., Duquesne, S. (eds.) AFRICACRYPT 23. LNCS, vol. 14064, pp. 317–343. Springer, Cham (Jul 2023)
13. Cong, K., Das, D., Park, J., Pereira, H.V.L.: SortingHat: Efficient private decision tree evaluation via homomorphic encryption and transciphering. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) ACM CCS 2022. pp. 563–577. ACM Press (Nov 2022)
14. D’Anvers, J.P., Pottier, X., de Ruijter, T., Verbaauwhede, I.: Head start: Digit extraction in TFHE from MSB to LSB. Cryptology ePrint Archive, Paper 2025/2012 (2025), <https://eprint.iacr.org/2025/2012>
15. Ducas, L., Micciancio, D.: Improved short lattice signatures in the standard model. In: Garay, J.A., Gennaro, R. (eds.) CRYPTO 2014, Part I. LNCS, vol. 8616, pp. 335–352. Springer, Berlin, Heidelberg (Aug 2014)
16. Ducas, L., Micciancio, D.: FHEW: Bootstrapping homomorphic encryption in less than a second. In: Oswald, E., Fischlin, M. (eds.) EUROCRYPT 2015, Part I. LNCS, vol. 9056, pp. 617–640. Springer, Berlin, Heidelberg (Apr 2015)
17. Gentry, C.: Computing on encrypted data (invited talk). In: Garay, J.A., Miyaji, A., Otsuka, A. (eds.) CANS 09. LNCS, vol. 5888, p. 477. Springer, Berlin, Heidelberg (Dec 2009)
18. Gentry, C., Halevi, S., Peikert, C., Smart, N.P.: Ring switching in BGV-style homomorphic encryption. In: Visconti, I., De Prisco, R. (eds.) SCN 12. LNCS, vol. 7485, pp. 19–37. Springer, Berlin, Heidelberg (Sep 2012)
19. Gentry, C., Sahai, A., Waters, B.: Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In: Canetti, R., Garay, J.A. (eds.) CRYPTO 2013, Part I. LNCS, vol. 8042, pp. 75–92. Springer, Berlin, Heidelberg (Aug 2013)
20. Guimarães, A., Borin, E., Aranha, D.F.: Revisiting the functional bootstrap in TFHE. IACR Transactions on Cryptographic Hardware and Embedded Systems **2021**(2), 229–253 (Feb 2021), <https://tches.iacr.org/index.php/TCHES/article/view/8793>
21. Guo, J., Sasaki, Y., Wang, L., Wu, S.: Cryptanalysis of HMAC/NMAC-Whirlpool. In: Sako, K., Sarkar, P. (eds.) ASIACRYPT 2013, Part II. LNCS, vol. 8270, pp. 21–40. Springer, Berlin, Heidelberg (Dec 2013)
22. Halevi, S., Shoup, V.: Algorithms in HELib. In: Garay, J.A., Gennaro, R. (eds.) CRYPTO 2014, Part I. LNCS, vol. 8616, pp. 554–571. Springer, Berlin, Heidelberg (Aug 2014)
23. Kang, J., Schild, L.: Pirouette: Query efficient single-server PIR. Cryptology ePrint Archive, Paper 2025/680, to appear in PoPETS 2026 (2025), <https://eprint.iacr.org/2025/680>
24. Karenin, A., Kirshanova, E., Nowakowski, J., Postlethwaite, E.W., Pulles, L.N., Virdia, F., Vié, P.: Cool + cruel = dual, and new benchmarks for sparse LWE. Cryptology ePrint Archive, Paper 2025/1002 (2025), <https://eprint.iacr.org/2025/1002>
25. Kluczniak, K., Schild, L.: Fdfb2: Full-domain functional bootstrapping with function amortization. In: Proceedings of the 13th Workshop on Encrypted Computing & Applied Homomorphic Computing. p. 13–25. WAHC ’25, Association for Computing Machinery, New York, NY, USA (2025), <https://doi.org/10.1145/3733811.3767310>
26. Kolmogorov, A.N.: On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition. Dokl. Akad. Nauk SSSR **114**(5), 953–956 (1957), <http://mi.mathnet.ru/dan22050>
27. Lee, K.H., Yoon, J.W.: Homomorphic field trace revisited : Breaking the cubic noise barrier. IACR Transactions on Cryptographic Hardware and Embedded Systems **2026**(1), 105–134 (Jan 2026), <https://tches.iacr.org/index.php/TCHES/article/view/12669>
28. Liu, Z., Micciancio, D., Polyakov, Y.: Large-precision homomorphic sign evaluation using FHEW/TFHE bootstrapping. In: Agrawal, S., Lin, D. (eds.) ASIACRYPT 2022, Part II. LNCS, vol. 13792, pp. 130–160. Springer, Cham (Dec 2022)

29. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. In: Gilbert, H. (ed.) EUROCRYPT 2010. LNCS, vol. 6110, pp. 1–23. Springer, Berlin, Heidelberg (May / Jun 2010)
30. Lyubashevsky, V., Peikert, C., Regev, O.: A toolkit for ring-LWE cryptography. In: Johansson, T., Nguyen, P.Q. (eds.) EUROCRYPT 2013. LNCS, vol. 7881, pp. 35–54. Springer, Berlin, Heidelberg (May 2013)
31. Ma, S., Huang, T., Wang, A., Xu, C., Wei, T., Wang, X.: Meta-PBS: Compact high-precision programmable bootstrapping. Cryptology ePrint Archive, Paper 2025/2284 (2025), <https://eprint.iacr.org/2025/2284>
32. Ma, S., Huang, T., Wang, A., Zhou, Q., Wang, X.: Fast and accurate: Efficient full-domain functional bootstrap and digit decomposition for homomorphic computation. IACR Transactions on Cryptographic Hardware and Embedded Systems **2024**(1), 592–616 (Dec 2023), <https://tches.iacr.org/index.php/TCHES/article/view/11263>
33. Micciancio, D., Polyakov, Y.: Bootstrapping in FHEW-like cryptosystems. In: WAHC ’21: Proceedings of the 9th on Workshop on Encrypted Computing & Applied Homomorphic Cryptography, Virtual Event, Korea, 15 November 2021. pp. 17–28. WAHC@ACM (2021)
34. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. In: Gabow, H.N., Fagin, R. (eds.) 37th ACM STOC. pp. 84–93. ACM Press (May 2005)
35. Wang, R., Wen, Y., Li, Z., Lu, X., Wei, B., Liu, K., Wang, K.: Circuit bootstrapping: Faster and smaller. In: Joye, M., Leander, G. (eds.) EUROCRYPT 2024, Part II. LNCS, vol. 14652, pp. 342–372. Springer, Cham (May 2024)